

STRINGS

- ❖ A **string** is a sequence (or suites) of characters, usually seen as a sentence or words.
- ❖ It is written in quotation marks (double quote), e.g.: "HELLO", "Give the value of X".
- ❖ It is considered a structured type (an array of characters): a one-dimensional array (vector) of which each element contains a single character.
 - Access to an element (character) is identical to that of vectors.
- ❖ A string variable can be declared in two different ways: a character vector or an abstract type called **String**.

STRINGS

	Character vector	String
Declaration	ch: array [100] of character	ch: string [100]
Reading	Do print("Give n between 1 et 100"); scan(n); while (n<0) or (n>100); For i←0 to n-1 do Begin print("Give a ieme charater"); scan (ch[i]); End _{For} ;	scan(ch); <i>/*the real length "n" is calculated because the string ends with a special null character '\0' (pronounced "backslash zero" or "null terminator") that indicates the end of the string. n←0 While (ch[n] != '\0') Do n++ */</i>

STRINGS

	Character vector	String											
Declaration	ch: array [100] of char	ch: string [100]											
Writing	For i←0 to n-1 do print (ch[i]);												
		print (ch)											
Example1 "Hello"	//Reading n←5; ch[0] = 'H'; ch[1] = 'e'; ch[2] = 'l'; ch[3] = 'l'; ch[4] = 'o'; <table><tr><td>H</td><td>e</td><td>l</td><td>l</td><td>o</td></tr></table> //printing Hello	H	e	l	l	o	//Reading ch ← "Hello » <table><tr><td>H</td><td>e</td><td>l</td><td>l</td><td>o</td><td>\0</td></tr></table> //printing Hello	H	e	l	l	o	\0
H	e	l	l	o									
H	e	l	l	o	\0								

STRINGS

❖ Assignments and comparisons are only done by for a single character.

❖ **Example**

1. p,q: char ; /* p and q are characters */
2. chaine1: array [10] of char; chaine2: string
3. p ← 'A' ; /* valid instruction */
4. chaine1 ← "Bonjour" ; /* INVALID instruction (A) */
5. chaine2 ← "Hello" ; /* valid instruction */
6. if (p = q) ; /* valid instruction */
7. if (chaine1 = chaine2) /* INVALID instruction (B) */

➤ To make the assignment (A) and the comparison (B), it would therefore be necessary to proceed character by character.

STRINGS

- In C language, a string is a vector of characters that can be declared in different ways: **char ch[Smax]**, **char ch []** or **char ***.
- There is no abstract type of **string**, but there is a library **<string.h>** that contains several modules for manipulating strings.
- In this course (or even the PWs of this semester), we will use only the declaration **char ch[Smax]** (where Smax is a constant), and we will return to the other notions (dynamic declaration or declaration as much as pointer and the modules of the **<string.h>** library) in semester 2.

STRINGS

- ❖ The `scanf` and `printf` formatted input-output functions can read and write strings with the format `%s`.
- ❖ The `scanf` function reads on the standard input a sequence of characters up to (and this is its main disadvantage) the encounter of a space spacing character, tab or line break, arranges it at the indicated address and completes it with a `\0` symbol.
- ❖ **For example:**

```
char s[20];  
scanf("%s",&s);  
printf("s = %s", s);
```

Si l'utilisateur saisi au clavier : `Bonjour`␣

il obtiendra l'affichage :

`s = Bonjour`

STRINGS

- ❖ The `scanf` and `printf` formatted input-output functions can read and write strings with the format `%s`.
- ❖ The `scanf` function reads on the standard input a sequence of characters up to (and this is its main disadvantage) the encounter of a space spacing character, tab or line break, arranges it at the indicated address and completes it with a `\0` symbol.
- ❖ **For example:**

```
char s[20];  
scanf("%s",&s);  
printf("s = %s", s);
```

Si maintenant l'utilisateur saisi au clavier : Bonne journée_

il obtiendra l'affichage :

s = Bonne

STRINGS

- ❖ The spacing character between **Bonne** and **journée** ends the reading. However, it is common to have to read a line, that is to say a string of characters containing several words spaced by spaces or tabs and ending with a line break character `\n`.
- ❖ To read a full line, you can use the `fgets` function, the prototype of which is as follows:

`char *fgets (char *s, int size, FILE *stream ) ;`

- ❖ It is actually a function that can be read from files but can be used to read from the standard input stream (`stdin`). This function reads at most `size-1` from the stream (which we can replace with `stdin`) and stores them in the character string `s` . Playback stops after EoF or a line break. A **null** character `\0` is added at the end of the string.

STRINGS

`char *fgets (char *s, int size, FILE *stream ) ;`

- ❖ It is actually a function that can be read from files but can be used to read from the standard input stream (stdin). This function reads at most size-1 from the stream (which we can replace with stdin) and stores them in the character string s . Playback stops after EoF or a line break. A null character\0 is added at the end of the string.
- ❖ For example:

```
char s[20];  
fgets(s,20,stdin);  
printf("s = %s", s);
```

La saisie au clavier de Bonne journée_ provoquera bien l'affichage

s = Bonne journée